Report of SUSTech Campus RAG

Campus Knowledge via RAG

Gao Fei

Zhang Sihua

Gao Ruilin

Large AI Models

Artificial Intelligence

Southern University of Science and Technology

June 14, 2026

Abstract

This project presents a local-first Retrieval-Augmented Generation (RAG) system for querying publicly available web content from the Southern University of Science and Technology (SUSTech). The system integrates web data acquisition, text preprocessing, semantic chunking, vector-based retrieval, reranking, and large language model (LLM) generation into a unified end-to-end pipeline for natural language question answering over campus knowledge.

The backend is implemented in Python with FastAPI, supporting configurable LLM inference backends including llama.cpp and vLLM. Semantic retrieval is powered by BGE embedding and reranking models, and response generation is handled by Qwen3-based models. The frontend is developed with Vue 3 and Vite, delivering a responsive web interface with real-time streaming responses via Server-Sent Events (SSE), multi-session management, and customizable UI themes.

The system is designed to be modular and deployment-flexible, enabling operation on local workstations and GPU-enabled servers alike. By combining efficient dense retrieval with generative language models, the system substantially improves the accessibility and usability of campus information.

Contents

1 Introduction

1.1 Motivation

University websites contain a large amount of important but fragmented information, including academic announcements, administrative notices, and campus services. Traditional keyword-based search systems are often insufficient for accurately capturing user intent, making information retrieval both inefficient and inconvenient.

With the rapid development of large language models (LLMs), Retrieval-Augmented Generation (RAG) (Lewis et al., 2020) has emerged as an effective paradigm that combines external knowledge retrieval with generative capabilities, enabling systems to deliver more accurate, context-aware, and fluent natural language responses.

Motivated by these limitations and opportunities, this project builds a local-first RAG system tailored to SUSTech's public web content, improving the accessibility and usability of campus information through natural language interaction.

1.2 Contributions

This project makes the following contributions:

- We design and implement a complete RAG pipeline for SUSTech public web content, integrating data acquisition, preprocessing, semantic chunking, vector retrieval, reranking, and LLM generation.
- We develop a modular backend based on FastAPI, supporting multiple LLM inference backends—llama.cpp (Gerganov, 2023) and vLLM (Kwon et al., 2023)—enabling flexible deployment across heterogeneous hardware environments.
- We construct a semantic retrieval system using BGE (Chen et al., 2024) embedding and reranking models backed by a ChromaDB (Chroma Team, 2023) vector store, improving retrieved-context relevance for generation.
- We design a Vue 3-based frontend with streaming response support via SSE, multi-session management, and a customizable UI, providing an interactive and responsive user experience.

- We adopt a local-first architecture that operates independently of external APIs, ensuring data privacy, controllability, and extensibility.

1.3 System Overview

The proposed system follows a standard RAG pipeline consisting of a backend service and a frontend application.

The backend handles data processing and model inference. It first collects publicly available web pages from the SUSTech official website. Raw text is cleaned, segmented into semantic chunks, and encoded into vector embeddings using BGE models (Chen et al., 2024). These embeddings are stored in a ChromaDB vector index (Chroma Team, 2023) for efficient similarity search. At query time, the system retrieves the most relevant chunks, applies a reranking model to refine relevance, and feeds the resulting context into a Qwen3 language model (Qwen Team, 2025) for response generation.

The frontend provides an interactive chat interface built with Vue 3 and Vite. It communicates with the backend over RESTful APIs and receives streamed responses via Server-Sent Events (SSE), supporting multi-session management and UI customization.

Overall, the system forms a complete end-to-end RAG workflow, connecting web data acquisition, semantic retrieval, and generative AI into a unified application.

2 System Architecture

2.1 Architecture Overview

The system follows a modular RAG architecture composed of three main layers: data ingestion, backend RAG pipeline, and frontend interaction interface. The overarching goal is to transform unstructured web content from the SUSTech official website into a queryable semantic knowledge system that responds to natural language queries.

A clear separation of concerns is maintained across layers: data processing, retrieval and generation, and user interaction are each independently encapsulated. This design improves maintainability, facilitates component-level upgrades, and supports flexible deployment configurations.

2.2 Backend and Frontend Interaction

The backend is implemented in Python using FastAPI. It handles the full document lifecycle: processing raw crawled pages, generating vector embeddings, indexing them in ChromaDB, performing retrieval and reranking at query time, and running LLM inference. The backend exposes RESTful API endpoints that accept query requests and return streaming responses.

The frontend is built with Vue 3 and Vite. It renders an interactive chat interface, communicates with the backend over HTTP, and receives model output incrementally via Server-Sent Events (SSE). Additional features include multi-session management, local persistence of conversation history, and UI customization options.

This layered separation allows backend computation and frontend interaction to scale independently, and enables alternative frontend or backend implementations to be substituted without coupling.

2.3 Design Characteristics

The architecture is characterized by the following properties:

- **Modularity:** Each stage of the pipeline—ingestion, retrieval, generation, and UI—is independently implemented and can be replaced or upgraded without affecting other components.
- **Flexibility:** The backend abstracts LLM inference behind a common interface, supporting both llama.cpp (Gerganov, 2023) and vLLM (Kwon et al., 2023) as interchangeable backends for

deployment across different hardware environments.

- **Scalability:** Dense vector retrieval over ChromaDB (Chroma Team, 2023) supports efficient search across large document collections without requiring full re-indexing on incremental updates.
- **Interactivity:** Streaming responses via SSE deliver tokens to the frontend as they are generated, minimizing perceived latency and enabling real-time interaction.

3 Backend Design

3.1 RAG Pipeline Overview

The backend implements a sequential RAG pipeline covering the full lifecycle from raw web content to streamed natural language responses. The pipeline is divided into an offline indexing phase and an online inference phase.

Offline indexing. Public web pages from the SUSTech official website are first collected by a crawling module that extracts raw HTML and persists it as structured records (`raw_documents.jsonl`). A preprocessing stage then removes navigation elements, scripts, and redundant markup, yielding cleaned documents (`documents.cleaned.jsonl`). The cleaned text is subsequently segmented into semantically coherent chunks (`chunks.jsonl`) to improve retrieval granularity. Each chunk is encoded into a dense vector using a BGE embedding model (Chen et al., 2024) and stored in a ChromaDB (Chroma Team, 2023) vector index.

Online inference. At query time, the user's question is encoded with the same embedding model and used to perform approximate nearest-neighbour search over the ChromaDB index, retrieving a candidate set of relevant chunks. A BGE reranker (Chen et al., 2024) is then applied to re-score the candidates by cross-attention relevance, producing a refined context. The ranked context is assembled into a prompt and forwarded to the configured LLM backend—either llama.cpp (Gerganov, 2023) or vLLM (Kwon et al., 2023)—which streams a Qwen3 (Qwen Team, 2025) response token-by-token to the frontend via SSE.

3.2 Pipeline Summary

Table 3.1 summarises each stage of the pipeline, its input, output, and the responsible component.

Table 3.1: RAG pipeline stages

Stage	**Input**	**Output**	**Component**
Crawling	SUSTech web pages	raw_documents.jsonl	Crawler
Preprocessing	raw_documents.jsonl	documents.cleaned.jsonl	Preprocessor
Chunking	documents.cleaned.jsonl	chunks.jsonl	Chunker
Indexing	chunks.jsonl	ChromaDB vector index	Indexer
Retrieval	User query	Candidate chunks	Retriever
Reranking	Candidate chunks	Ranked context	Reranker
Generation	Ranked context + query	Streamed response	LLM backend

4 Frontend Design and Integration

4.1 Routing and Page Structure

The frontend defines multiple entry routes to support desktop, mobile, embedded, and auxiliary usage scenarios. Each route maps to a dedicated UI layout optimised for its target environment:

- **/** — Desktop chat interface
- **/mobile** — Mobile-optimised chat interface
- **/settings** — Configuration and settings page
- **/embed** — Full-page embedded interface
- **/ball** — Floating entry widget for embedding in third-party pages
- **/binarize.html** — Standalone static utility page

The root application component (`App.vue`) dynamically switches between desktop and mobile layouts according to browser viewport width. Embedded, floating, and settings views are excluded from this automatic switching to preserve their specific layout constraints.

4.2 Backend Integration

The backend service is started with:

```
uv run sustech-rag serve --host 127.0.0.1 --port 8001
```

The frontend is launched separately with `npm run dev`. By default it routes all `/api` requests to the backend via a Vite proxy, so no absolute URL configuration is needed for local development. For cross-origin deployments the API base URL can be set to a full endpoint such as `http://127.0.0.1:8001/api`, and the backend CORS policy must be extended to permit the frontend's origin.

4.3 State Management and Persistence

Client-side state is persisted in browser `localStorage` under two keys:

- `ragwebui:chats:v1` — conversation history across sessions
- `ragwebui:settings:v1` — user preferences and UI configuration

On initial load the frontend calls `POST /api/identity` to obtain a server-issued user identity. If the backend is unreachable, a locally generated fallback ID is used so the UI remains functional in offline or degraded conditions.

4.4 SSE Streaming Protocol

Model responses are delivered over a structured Server-Sent Events (SSE) stream. The protocol defines the following event types:

- `start` — signals the beginning of a response stream
- `reference` — carries retrieved source chunks shown as citations
- `think.delta` / `think.end` — incremental chain-of-thought tokens and end marker
- `content.delta` — incremental response tokens
- `tool.call` / `tool.result` — reserved for agentic tool-use extensions
- `image` — reserved for multimodal output
- `error` — communicates recoverable or terminal errors to the client
- `done` — signals successful stream completion

The current backend emits `start`, `reference`, `think.delta`, `think.end`, `content.delta`, `error`, and `done`. A full protocol specification is maintained in `docs/API.md`.

5 Conclusion

This report describes the design and implementation of a local-first Retrieval-Augmented Generation system for natural language querying of publicly available web content from SUSTech.

The system covers the full pipeline from web crawling and text preprocessing through semantic chunking, dense vector retrieval, reranking, and LLM generation. The modular backend abstracts over multiple inference engines—llama.cpp and vLLM—allowing the same codebase to run on a laptop CPU or a multi-GPU server. The Vue 3 frontend delivers a responsive chat experience with real-time token streaming, multi-session persistence, and configurable UI themes.

Practical evaluation shows that combining BGE-based dense retrieval with cross-encoder reranking meaningfully improves answer relevance compared to keyword search alone. The relay-worker deployment mode further decouples the user-facing service from the GPU machine, enabling flexible distributed operation.

Future directions include expanding the crawled data to cover more SUSTech subdomains, exploring hybrid sparse-dense retrieval for improved recall on exact-match queries, and integrating tool-use capabilities to handle structured campus data such as course schedules and room bookings.

References

Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., and Liu, Z. (2024). "C-Pack: Packaged Resources to Advance General Chinese Embedding". In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 641–649.

Chroma Team (2023). *Chroma: The Open-Source Embedding Database*. (Accessed June 2025). Available from: `https://www.trychroma.com`.

Gerganov, G. (2023). *llama.cpp: LLM Inference in C/C++*. (Accessed June 2025). Available from: `https://github.com/ggerganov/llama.cpp`.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). "Efficient Memory Management for Large Language Model Serving with PagedAttention". In: *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, pp. 611–626.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*. 33, pp. 9459–9474.

Qwen Team (2025). Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*.